School of Information and Computer Technology
Sirindhorn International Institute of Technology, Thammasat University
CSS326 Database Programming Laboratory

Laboratory #11: Stored Routines, Triggers & NoSQL

Objectives:

- To write stored procedures (with IN and OUT parameters) and stored functions, and call them.
- To create triggers that run automatically when rows are inserted or deleted.
- To call stored routines from a Node.js web app and handle the errors they raise.
- To store and query semi-structured data with the JSON column type.
- To use MySQL as a document store with the X DevAPI.

1. Recap and Goal

In Lab 9 you made multi-step actions safe with transactions, and in Lab 10 you made queries fast with indexes and safe with placeholders and least privilege. In all of these labs, every rule of the application, such as “enrolling takes a seat”, lived in your Node.js code; the database only stored and returned data. This lab explores features that live **inside MySQL itself**. The first half covers stored routines: procedures, functions and triggers. The second half covers MySQL’s NoSQL side: the JSON column type and the X DevAPI document store. We use the students, courses and enrolments schema from Lab 9, including the `seats` column, and a ready-made web app that shows the effect of everything you write.

2. Logic in the Database

A **stored routine** is code saved inside MySQL and run by the server when asked. The benefits are reuse (the same logic can be called from your API, from Workbench or from any other client) and fewer round-trips (a multi-step job runs next to the data in a single call). There are three kinds: **procedures**, which do something and are run with CALL; **functions**, which return one value and can be used inside a query; and **triggers**, which nobody calls: they run by themselves when a row changes.

3. Setup: the lab11 Database

Run `sql/lab11_setup.sql` from the exercise starter once in MySQL Workbench. It creates a fresh `lab11` database with the Lab 3 data, the UNIQUE key and foreign keys from Lab 9, and the `seats` column from Lab 9, already synced so that seats means **free** seats (capacity 30 minus the students already enrolled). Running the script again resets everything, including any procedures, functions and triggers you have written, which is handy if you want to start over.

4. Stored Procedures

A procedure is a named block of SQL that you run with CALL. Parameters declared `IN` pass values in. Because the body contains its own semicolons, you temporarily change the statement terminator with `DELIMITER //`, so that MySQL Workbench does not stop at the first inner semicolon; afterwards you switch it back with `DELIMITER ;`.

```
DELIMITER //
CREATE PROCEDURE enrol_student(IN p_student BIGINT, IN p_course INT)
BEGIN
  INSERT INTO enrolments (student_id, course_id) VALUES (p_student, p_course);
END //
DELIMITER ;

CALL enrol_student(3, 1);      -- student 3 joins course 1
```

To change a procedure, drop it and create it again: `DROP PROCEDURE IF EXISTS enrol_student;` (the exercise scripts already start with these lines). Parameter names start with `p_` so they can never be confused with column names.

5. Parameters: IN and OUT

An `IN` parameter carries a value into the routine; an `OUT` parameter hands a result back to the caller. Inside the body, `SELECT … INTO` stores a query result in a variable. The caller passes a session variable (written `@name`) for the OUT value and reads it afterwards with SELECT.

```
DELIMITER //
CREATE PROCEDURE count_students(IN p_course INT, OUT p_total INT)
BEGIN
  SELECT COUNT(*) INTO p_total FROM enrolments WHERE course_id = p_course;
END //
DELIMITER ;

CALL count_students(1, @n);
SELECT @n;                     -- read the OUT value
```

6. Stored Functions

A function **returns a single value** and can be used directly inside SELECT, WHERE or ORDER BY, just like a built-in function. Declare what it RETURNS, compute the value in the body, and RETURN it. `DECLARE` creates a local variable inside BEGIN … END.

```
DELIMITER //
CREATE FUNCTION course_count(p_student BIGINT) RETURNS INT
READS SQL DATA
BEGIN
  DECLARE n INT;
  SELECT COUNT(*) INTO n FROM enrolments WHERE student_id = p_student;
  RETURN n;
END //
DELIMITER ;

SELECT Id, name, course_count(Id) AS courses FROM students;
```

> **Note**
>
> A default MySQL 8 installation has binary logging switched on, and then every function must say what kind of work it does. `READS SQL DATA` means “this function only reads data”, which is true here. Without it you get **Error 1418: This function has none of DETERMINISTIC, NO SQL, or READS SQL DATA in its declaration**. `DETERMINISTIC` (same inputs always give the same result) would also be accepted, but it is not really true for a function that counts rows that change.

7. Triggers: Automatic Actions

A trigger is code that runs by itself whenever a row is inserted, updated or deleted on a table. It fires `BEFORE` or `AFTER` the change, once `FOR EACH ROW`. Inside the trigger, `NEW` is the new version of the row (in INSERT and UPDATE triggers) and `OLD` is the previous version (in UPDATE and DELETE triggers). Triggers are excellent for keeping data in step automatically, such as the free seats of a course, or for writing an audit log. Use them with care: the action is invisible in the application code, so **document every trigger** you add.

8. A Trigger that Guards the Seats

The trigger below takes one seat whenever an enrolment is inserted. It reuses the guard from Lab 9: the UPDATE only changes a row while `seats > 0`. `ROW_COUNT()` returns how many rows the previous statement changed; if it is 0 the course was full, and `SIGNAL` raises an error with our own message. A second trigger gives the seat back when an enrolment is deleted, using `OLD`.

```
DELIMITER //
CREATE TRIGGER after_enrolment_insert
AFTER INSERT ON enrolments
FOR EACH ROW
BEGIN
  UPDATE courses SET seats = seats - 1
   WHERE id = NEW.course_id AND seats > 0;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'course is full';
  END IF;
END //

CREATE TRIGGER after_enrolment_delete
AFTER DELETE ON enrolments
FOR EACH ROW
BEGIN
  UPDATE courses SET seats = seats + 1 WHERE id = OLD.course_id;
END //
DELIMITER ;
```

Why is no transaction needed here? An INSERT and the triggers it fires form **one statement**, and in InnoDB a statement succeeds or fails as a whole. When the trigger SIGNALs, the INSERT that fired it is undone too, so a full course never receives an enrolment. In Lab 9, Node had to wrap two statements in a transaction to get the same guarantee. SQLSTATE `45000` means “unhandled user-defined exception”; Workbench shows it as **Error Code: 1644. course is full**.

9. Calling Routines from Node.js

From mysql2 a stored procedure is just another statement: run it with CALL and pass the arguments through `?` placeholders as usual (Lab 10: never glue values into the SQL string). A function can be used inside an ordinary SELECT. When a trigger SIGNALs, mysql2 rejects the promise with an error whose `sqlState` is `'45000'` and whose `sqlMessage` is your message, so the controller can turn it into a clear **409 Conflict**.

```
// POST /students/:id/enrolments
try {
  await pool.execute('CALL enrol_student(?, ?)', [studentId, courseId]);
  res.status(201).json({ ok: true, student_id: studentId, course_id: courseId });
} catch (err) {
  if (err.sqlState === '45000')                       // our SIGNAL
    return res.status(409).json({ error: err.sqlMessage });   // 'course is full'
  if (err.code === 'ER_DUP_ENTRY')
    return res.status(409).json({ error: 'already enrolled in this course' });
  if (err.code === 'ER_NO_REFERENCED_ROW_2')
    return res.status(404).json({ error: 'student or course not found' });
  res.status(500).json({ error: err.message });
}

// GET /students - the function inside a SELECT
const [rows] = await pool.execute(
  `SELECT Id AS id, name, major, course_count(Id) AS courses
     FROM students ORDER BY Id`);
```

10. One Rule, One Home

Routines keep logic close to the data and reusable across clients, which suits shared, data-heavy rules. The trade-off is that logic in the database is harder to version, test and debug than code in your app, so many web applications keep most logic in Node and use routines or triggers for a few specific jobs. Whatever you choose, give each rule **one** home. The exercise starter shows what happens otherwise: its enrol route is still your Lab 9 transaction, which updates `seats` in Node. Once your trigger exists, the INSERT takes a seat as well, so one enrolment costs **two** seats. The web page's log makes this visible ("seats 28 → 26 (−2), expected −1"), and Exercise 3 fixes it by letting Node call the procedure and leaving the seats to the trigger.

Procedure, function or trigger? The three kinds of routine look alike but do different jobs:

	Stored procedure	Stored function	Trigger
How it runs	You CALL it	Inside a query (SELECT, WHERE, ORDER BY)	By itself, on INSERT / UPDATE / DELETE
Gives back	OUT parameters or result rows	Exactly one value (RETURNS)	Nothing: it reacts to the change
Changes data?	Yes: that is its main job	Should not (READS SQL DATA)	Yes: keeps other rows in step
Own transaction?	Yes: START TRANSACTION … COMMIT	No	No: it is part of the statement that fired it
From Node.js	`CALL enrol_student(?, ?)`	`SELECT …, course_count(Id) …`	Never called: fires on your INSERT
In this lab	`enrol_student`, `count_students`	`course_count`	`after_enrolment_insert` / `_delete`

Where should the seat rule live? Enrolling must take a seat and insert a row, all-or-nothing. There are three good homes for that rule. In Lab 9 it lived in Node, as a transaction on one borrowed connection. It can also live in a procedure that runs its own transaction, so one CALL does both steps on the server; the EXIT HANDLER rolls back if any statement fails, and RESIGNAL passes the error on to Node:

```
CREATE PROCEDURE enrol_student_tx(IN p_student BIGINT, IN p_course INT)
BEGIN
  DECLARE EXIT HANDLER FOR SQLEXCEPTION
  BEGIN
    ROLLBACK;      -- undo the seat
    RESIGNAL;      -- and pass the error on to the caller
  END;
  START TRANSACTION;
  UPDATE courses SET seats = seats - 1 WHERE id = p_course AND seats > 0;
  IF ROW_COUNT() = 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'course is full';
  END IF;
  INSERT INTO enrolments (student_id, course_id) VALUES (p_student, p_course);
  COMMIT;
END
```

In this lab the rule lives in the trigger, and `enrol_student` only inserts. Each home has a trade-off. Node code is the easiest to test and version, but every other client must repeat the rule. A procedure gives every caller the same rule in one CALL, but a plain INSERT still skips it. A trigger cannot be skipped, not even by an INSERT typed in Workbench, but it is hidden from anyone who reads only the app code. All three are atomic. The one thing you must not do is use two homes at once: if `enrol_student` also updated `courses`, every enrolment would cost two seats.

11. NoSQL with MySQL: the JSON Type

Not all data fits neat columns: sometimes the fields vary from row to row. MySQL has a `JSON` column type, so you can store a whole flexible object in one column and keep relational tables and document-style data in the same database. Declare a JSON column and build a value with `JSON_OBJECT` and `JSON_ARRAY`, or store a valid JSON string directly. MySQL rejects a value that is not valid JSON.

```
ALTER TABLE courses ADD COLUMN details JSON;

UPDATE courses
   SET details = JSON_OBJECT('room','BKD 3201', 'day','Mon', 'time','09:00',
                 'online', false, 'tags', JSON_ARRAY('core','programming'))
 WHERE id = 1;

-- a JSON literal works too; this row has an extra field (zoom) the others don't
UPDATE courses
   SET details = '{"room": "Online", "day": "Wed", "time": "10:00",
                  "online": true, "zoom": "css-stat", "tags": ["math"]}'
 WHERE id = 4;
```

12. Querying JSON

Pull fields out of a JSON column with path expressions. `details->>'$.room'` follows the path `$.room` into the document and returns the value as plain **text**; `details->'$.tags'` returns it as a **JSON value** (here an array, which mysql2 turns into a JavaScript array); and `JSON_EXTRACT(details, '$.tags')`

is the long form of `->`. A field that is missing in a row is simply NULL. You can use these expressions in the SELECT list and in WHERE, and the value to compare with still goes in as a `?` parameter.

```
SELECT title, details->>'$.room' AS room, details->>'$.day' AS day FROM courses;

SELECT title FROM courses WHERE details->>'$.day' = 'Mon';

SELECT title FROM courses WHERE JSON_CONTAINS(details->'$.tags', '"core"');

// in the controller: GET /courses?day=Mon
let sql = "SELECT id, title, details->>'$.room' AS room FROM courses";
const params = [];
if (req.query.day) {
  sql += " WHERE details->>'$.day' = ?";
  params.push(req.query.day);
}
const [rows] = await pool.execute(sql, params);
```

13. The Document Store: X DevAPI

MySQL can also behave like a document database. Through the **X DevAPI** (the X Protocol on port **33060**, with the `@mysql/xdevapi` package) you store JSON documents in **collections** without defining a schema, and work with them using `add()` and `find()` instead of SQL. Two documents in the same collection may carry different fields. This is MySQL's NoSQL face: the same server, used as a document store. It suits prototypes and data whose shape is still changing, such as free-form course reviews.

```
const mysqlx = require('@mysql/xdevapi');
const session = await mysqlx.getSession({ host: process.env.DB_HOST,
  port: 33060, user: process.env.DB_USER, password: process.env.DB_PASS });
const reviews = await session.getSchema('lab11')
  .createCollection('course_reviews', { reuseExisting: true });

await reviews.add({ course_id: 1, student: 'Ploy', stars: 5, tags: ['labs'] })
  .execute();
const result = await reviews.find('course_id = :c').bind('c', 1).execute();
console.log(result.fetchAll());
await session.close();
```

Port 33060. The X DevAPI does not use the normal MySQL port. It talks to the server through the **X Protocol** on port **33060**, while Workbench, `mysql2` and the rest of the app use **3306**. It is the same server and the same login, just a second door. In the Lab 11 app the port comes from `DB_X_PORT=33060` in `.env`, which `controllers/reviewController.js` reads. The X Protocol is built into MySQL 8 and switched on by default; you can check it in Workbench:

```
SHOW VARIABLES LIKE 'mysqlx_port';   -- expect 33060
SHOW VARIABLES LIKE 'mysqlx';        -- expect ON
```

If the app reports `ECONNREFUSED 127.0.0.1:33060`, the usual causes are:

- **Windows installer:** the "Enable X Protocol / MySQL as a Document Store" option was unticked during installation. Re-run MySQL Installer, choose **Reconfigure** for the server and tick it.

- **Firewall:** port 33060 is blocked. This only matters when MySQL runs on another machine; on localhost it is normally fine.
- **Custom port:** `mysqlx_port` was changed. Put the value the query shows into `DB_X_PORT`.

Exercise 5 is optional, so if the X Protocol is not available you can skip it; the rest of the app does not depend on port 33060.

14. When to Use What

Reach for ordinary columns first: structured data with fixed fields belongs in proper columns, which are easy to index, constrain and join. Use a JSON column for genuinely variable or sparse attributes that differ across rows, accepting that JSON fields are harder to index and validate. A common, sensible pattern is to keep the core fields as columns and a single JSON column for the long tail of extras. The document store is handy when the shape is still changing or you want a quick, schema-free space next to your tables.

15. Good Habits

- Use stored routines for shared, data-heavy rules, but keep most application logic in Node, where it is easy to test and version.
- Give each rule one home: if a trigger moves the seats, Node must not move them as well.
- Document every trigger: a hidden automatic action surprises anyone who reads only the app code.
- Add `READS SQL DATA` (or `DETERMINISTIC` when it is true) to every function.
- Prefer real columns for structured data and JSON for the genuinely variable parts.
- Keep parameterising values with `?`, even when calling procedures or filtering on JSON fields.

16. Lab Exercises (complete during class)

Getting started

Download and unzip **Lab11_Exercise_Starter.zip**. It contains:

```
Lab11_Exercise_Starter/
  .env                      YOUR settings: DB_USER, DB_PASS, ... (edit this)
  .env.example              shareable template of .env
  db.js                     connection pool (reads .env with dotenv)
  server.js                 Express app + front-end on http://localhost:3000
  public/index.html         the Lab 11 web app (courses, students, enrol, reviews)
  routes/                   courses.js, students.js
  controllers/              course-, student-, enrolment- and reviewController.js
  sql/lab11_setup.sql       creates the lab11 database (Section 3)
  sql/exercise1.sql         Exercise 1 script with TODOs
  sql/exercise2.sql         Exercise 2 script with TODOs
  sql/exercise4.sql         Exercise 4 script with TODOs
  requests.http             ready-made test requests
```

- In Workbench, run `sql/lab11_setup.sql` once.
- Open the **`.env`** file and change `DB_USER` and `DB_PASS` to your own MySQL login. `DB_NAME` is already `lab11`. As in Lab 10, `db.js` reads these values with `dotenv` (`process.env.DB_USER`, …), and so does the document store in Exercise 5. `.env` is listed in `.gitignore`; `.env.example` is the template you can share.

Then open a terminal **in the project folder** (in VS Code: Terminal → New Terminal) and run the two npm commands below.

```
npm install      # once: downloads express, mysql2, dotenv and @mysql/xdevapi
npm run dev      # starts the server
                 #   > node --watch server.js
                 #   Lab 11 server on http://localhost:3000
                 # Ctrl + C stops the server
```

`npm install` reads the dependencies listed in `package.json` and downloads them into `node_modules`. `@mysql/xdevapi` is listed as optional, so the app still starts if it cannot be installed (only Exercise 5 needs it). `npm run dev` runs `node --watch server.js`, which **restarts the server every time you save a .js file**. Leave the terminal open while you work; press **Ctrl + C** to stop the server.

- Open **http://localhost:3000** in your browser. The page has five panels: **Courses** (free seats, plus room, day and tags from JSON and a day filter), **Students** (with the course count from your function), **Enrol / cancel** for one student, **Course reviews** (document store) and a **Responses** log.
- Everything works from the start, but parts are empty until you finish an exercise; a yellow note in each panel tells you which one. After every enrol or cancel the log shows how the course's free seats changed, and flags it in red when the change is not exactly −1 or +1.
- You can also test with `requests.http` (VS Code REST Client) or Postman.

If something fails, read the first error line (in the terminal, in the Responses log, or in Workbench's Output panel):

Error message	What it means / what to do
`Access denied for user 'root'`	Wrong login: set your own `DB_USER` / `DB_PASS` in `.env`.
`Access denied for user ''@'localhost'`	`.env` was not found: it must be named exactly `.env` (not `.env.txt`), sit in the project folder, and be saved.
`Unknown database 'lab11'`	Run `sql/lab11_setup.sql` in Workbench first.
`FUNCTION lab11.course_count does not exist`	Exercise 3 uses the function from Exercise 1: create it first.
`Error 1418 … READS SQL DATA …`	Add `READS SQL DATA` after `RETURNS INT` (Section 6).
`Error 1644: course is full`	Not a bug: your trigger SIGNALled. In Node, answer 409.
`ECONNREFUSED …:33060`	The X Protocol is off: Exercise 5 is optional.
`EADDRINUSE … :3000`	The server is already running in another terminal; stop it with Ctrl + C.

Exercise 1: A procedure and a function. Open `sql/exercise1.sql` in Workbench. Write the procedure `enrol_student(p_student, p_course)` that inserts an enrolment, and CALL it to enrol student 3 in course 1. Write the function `course_count(p_student)` (remember `READS SQL DATA`) and use it in a SELECT that lists every student with their number of courses. Check the numbers against the plain GROUP BY query in the script. Finally, write `count_students(IN p_course, OUT p_total)`, CALL it with `@n` and read the value back.

```
CREATE PROCEDURE enrol_student(IN p_student BIGINT, IN p_course INT)
BEGIN
  -- TODO: INSERT INTO enrolments (student_id, course_id) VALUES (...);
END //
```

Exercise 2: Triggers keep the seats in step. Open `sql/exercise2.sql`. Step 1 re-syncs the free seats (Exercise 1 inserted an enrolment without taking a seat). Create `after_enrolment_insert`, which takes a seat with a guarded UPDATE and SIGNALs `'course is full'` when `ROW_COUNT() = 0`, and `after_enrolment_delete`, which gives the seat back. Test both with INSERT and DELETE in Workbench, and confirm that an insert into a full course fails with Error 1644 and leaves no enrolment behind. Then enrol a student **from the web page**: the seats drop by 2. Note in a comment why; Exercise 3 fixes it.

```
CREATE TRIGGER after_enrolment_insert AFTER INSERT ON enrolments
FOR EACH ROW
BEGIN
  -- TODO: UPDATE courses SET seats = seats - 1
  --        WHERE id = NEW.course_id AND seats > 0;
  -- TODO: IF ROW_COUNT() = 0 THEN
  --         SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'course is full';
  --       END IF;
END //
```

Exercise 3: Use the routines from the web app. In `controllers/enrolmentController.js`, replace the Lab 9 transaction in `enrol` with one call to your procedure, and map `err.sqlState === '45000'` to 409 (keep 409 for duplicates and 404 for unknown students). In `unenrol`, keep only the DELETE: the trigger gives the seat back. In `controllers/studentController.js`, add `course_count(Id) AS courses` to the SELECT. Reload the page: the Courses column is filled, and every enrol or cancel now changes the seats by exactly −1 or +1. Set a course to 0 seats in Workbench and check that enrolling answers **409 course is full**.

```
await pool.execute('CALL enrol_student(?, ?)', [studentId, courseId]);
res.status(201).json({ ok: true, student_id: studentId, course_id: courseId });
```



Exercise 4: JSON in practice. In `sql/exercise4.sql`, add the `details` JSON column, store an object for every course (course 1 and course 4 are done for you), and query each course's room and day, then only the Monday courses. Then, in `controllers/courseController.js`, return `room`, `day`, `time` and `tags` with `->>` and `->`, and support `GET /courses?day=Mon` with a `?` parameter. The Room, Day / time and Tags columns appear in the page, and the day filter works.

```
SELECT id, title, credit, seats,
       details->>'$.room' AS room, details->>'$.day' AS day,
       details->>'$.time' AS time, details->'$.tags' AS tags
  FROM courses
-- + " WHERE details->>'$.day' = ?"  when req.query.day is given
```

Exercise 5: Course reviews in a collection (if time). In `controllers/reviewController.js` (the login already comes from `.env`, with `DB_X_PORT=33060`), implement `list` with `find('course_id = :c').bind('c', courseId)` and `add` with `add(doc)`. Store `tags` only when the user typed some, so documents in the collection have different shapes. Add two reviews from the page, and note one situation where a document store fits better than a table.

17. Where We're Going Next

You have now seen the full breadth of MySQL: relational, programmable and document-style. That completes the technical labs, and you have every tool your course project needs. **Lab 12** is the Final Project Presentation: bring together your schema, SQL, API, transactions, performance and security work, and these advanced features to show what you have built.